

1 IPsec

1.1 IPsec with IKEv1

1.1.1 Setup

This experiment followed the IPsec and DMVPN lab guide topology [1]. The same base topology as in the previous GRE tunneling exercise was used. The objective was to replace the GRE tunnel between R1 and R2 with an IPsec tunnel using IKEv1. The public network was used only to provide connectivity between the two tunnel endpoints, while the private networks represented the two sites of the organization.

Table 1: Addressing plan used in the IPsec/IKEv1 simulation.

Device	Interface	Address	Role
R1	G0/0	200.1.1.1/24	Public network, site 1
R1	G0/1	192.168.2.1/24	Private network, site 1
R1	Lo0	1.1.1.1/32	IPsec tunnel endpoint, site 1
R1	Tunnel0	Unnumbered G0/1	IPsec tunnel interface
RA	F0/0	200.1.1.10/24	Public transit network
RA	F0/1	200.2.2.10/24	Public transit network
R2	G0/0	200.2.2.2/24	Public network, site 2
R2	G0/1	192.168.3.2/24	Private network, site 2
R2	Lo0	2.2.2.2/32	IPsec tunnel endpoint, site 2
R2	Tunnel0	Unnumbered G0/1	IPsec tunnel interface
R3	F0/0	192.168.2.3/24	Private network, site 1
R3	F0/1	192.168.1.3/24	Private network, site 1
R4	F0/0	192.168.3.4/24	Private network, site 2
R4	F0/1	192.168.4.4/24	Private network, site 2
PC1	Eth0	192.168.1.100/24	End host, site 1
PC2	Eth0	192.168.4.100/24	End host, site 2

1.1.2 IKEv1 with pre-shared key and AH

In the first part of the experiment, IPsec was configured between R1 and R2 using IKEv1 with a pre-shared key. The ISAKMP policy used SHA for hashing, AES-256 for encryption, Diffie-Hellman group 5 and a lifetime of 86400 seconds. The pre-shared key used by both routers was **saar**. For IPsec Phase 2, the transform set was configured with AH using **ah-sha-hmac**.

The tunnel interface was configured with `tunnel mode ipsec ipv4` and protected with the IPsec profile created previously. The OSPF process of the private organization network was then enabled over the tunnel interface.

Connectivity between IPsec endpoints

Before testing the communication between the end hosts, connectivity between the loopback interfaces of R1 and R2 was verified. These loopback interfaces are used as the IPsec tunnel endpoints.

```

R1#ping 2.2.2.2 source 1.1.1.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 2.2.2.2, timeout is 2 seconds:
Packet sent with a source address of 1.1.1.1
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 12/18/20 ms

```

Figure 1: Connectivity test between the IPSec endpoints.

As shown in Figure 1, R1 was able to reach the loopback address of R2 using 1.1.1.1 as the source address.

This confirms that the public underlay network was correctly configured and that R1 could reach R2 before establishing end-to-end communication through IPSec.

Tunnel interface status

After configuring the IPSec tunnel, the state of the interfaces was checked with the command `show ip interface brief`.

```

R1#show ip interface brief
Interface                IP-Address      OK? Method Status      Prot
GigabitEthernet0/0       200.1.1.1       YES manual up          up
GigabitEthernet0/1       192.168.2.1     YES manual up          up
GigabitEthernet0/2       unassigned      YES unset   administratively down down
GigabitEthernet0/3       unassigned      YES unset   administratively down down
Loopback0                 1.1.1.1         YES manual up          up
Tunnel0                   192.168.2.1     YES TFTP   up          up

```

Figure 2: Interface status on R1 after configuring the IPSec tunnel.

The output shows that the physical interfaces, the loopback interface and the tunnel interface were all operational. In particular, **Tunnel0** was in the **up/up** state.

This confirms that the tunnel interface was active and ready to transport traffic between the two sites.

IKEv1 Security Association

The IKEv1 Security Association was verified using the command `show crypto isakmp sa`.

```

R1#show crypto isakmp sa
IPv4 Crypto ISAKMP SA
dst          src          state          conn-id status
1.1.1.1      2.2.2.2      QM_IDLE        1001 ACTIVE
2.2.2.2      1.1.1.1      QM_IDLE        1002 ACTIVE
IPv6 Crypto ISAKMP SA

```

Figure 3: IKEv1 Security Association established between R1 and R2.

The output shows two active ISAKMP Security Associations between 1.1.1.1 and 2.2.2.2. Both entries were in the **QM_IDLE** state.

The **QM_IDLE** state confirms that the IKEv1 negotiation was completed successfully and that the IPSec tunnel was ready to protect traffic.

IPSec Security Association

The IPSec Security Association was then checked with the command `show crypto ipsec sa`.

```
R1#show crypto ipsec sa

interface: Tunnel0
  Crypto map tag: Tunnel0-head-0, local addr 1.1.1.1

protected vrf: (none)
local ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
remote ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
current_peer 2.2.2.2 port 500
  PERMIT, flags={origin_is_acl,}
  #pkts encaps: 38, #pkts encrypt: 38, #pkts digest: 38
  #pkts decaps: 37, #pkts decrypt: 37, #pkts verify: 37
  #pkts compressed: 0, #pkts decompressed: 0
  #pkts not compressed: 0, #pkts compr. failed: 0
  #pkts not decompressed: 0, #pkts decompress failed: 0
  #send errors 0, #recv errors 0

  local crypto endpt.: 1.1.1.1, remote crypto endpt.: 2.2.2.2
  plaintext mtu 1456, path mtu 1500, ip mtu 1500, ip mtu idb GigabitEthernet0/0
  current outbound spi: 0x80A5B801(2158344193)
  PFS (Y/N): N, DH group: none

inbound esp sas:

inbound ah sas:
  spi: 0xBAF2D51F(3136476447)
  transform: ah-sha-hmac ,
  in use settings ={Tunnel, }
  conn id: 5, flow_id: SW:5, sibling_flags 80004050, crypto map: Tunnel0-head-0
  sa timing: remaining key lifetime (k/sec): (4258874/3428)
  replay detection support: Y
  Status: ACTIVE(ACTIVE)

inbound pcp sas:

outbound esp sas:

outbound ah sas:
  spi: 0x80A5B801(2158344193)
  transform: ah-sha-hmac ,
  in use settings ={Tunnel, }
  conn id: 6, flow_id: SW:6, sibling_flags 80004050, crypto map: Tunnel0-head-0
  sa timing: remaining key lifetime (k/sec): (4258874/3428)
  replay detection support: Y
  Status: ACTIVE(ACTIVE)

outbound pcp sas:

R1#
```

Figure 4: IPSec Security Association statistics on R1.

The most relevant fields in this output are the packet counters. After generating traffic between PC1 and PC2, the values of `#pkts encaps` and `#pkts decaps` increased, confirming that packets were being protected and transported through IPSec.

End-to-end connectivity test

After confirming that the tunnel was active, a ping was performed from PC1 to PC2.

```

PC1> ping 192.168.4.100

84 bytes from 192.168.4.100 icmp_seq=1 ttl=60 time=49.896 ms
84 bytes from 192.168.4.100 icmp_seq=2 ttl=60 time=35.330 ms
84 bytes from 192.168.4.100 icmp_seq=3 ttl=60 time=35.585 ms
84 bytes from 192.168.4.100 icmp_seq=4 ttl=60 time=45.812 ms
84 bytes from 192.168.4.100 icmp_seq=5 ttl=60 time=45.527 ms

```

Figure 5: Successful ping from PC1 to PC2 through the IPsec tunnel.

The ping was successful, as shown below.

This result confirms that the two private sites were able to communicate through the IPsec tunnel.

Analysis of AH packets

While the ping between PC1 and PC2 was running, a Wireshark capture was performed in the public network. The ICMP traffic was observed inside AH packets.

No.	Time	Source	Destination	Protocol	Length	Info
26	30.404290	192.168.4.100	192.168.1.100	ICMP	142	Echo (ping) reply id=0x9b31, seq=2/512, ttl=62 (request in 25)
27	30.897235	192.168.3.2	224.0.0.5	OSPF	138	Hello Packet
28	31.421080	192.168.1.100	192.168.4.100	ICMP	142	Echo (ping) request id=0x9c31, seq=3/768, ttl=62 (reply in 29)
29	31.450551	192.168.4.100	192.168.1.100	ICMP	142	Echo (ping) reply id=0x9c31, seq=3/768, ttl=62 (request in 28)
31	32.467730	192.168.1.100	192.168.4.100	ICMP	142	Echo (ping) request id=0x9d31, seq=4/1024, ttl=62 (reply in 32)
32	32.496988	192.168.4.100	192.168.1.100	ICMP	142	Echo (ping) reply id=0x9d31, seq=4/1024, ttl=62 (request in 31)
34	33.514216	192.168.1.100	192.168.4.100	ICMP	142	Echo (ping) request id=0x9e31, seq=5/1280, ttl=62 (reply in 35)
35	33.543497	192.168.4.100	192.168.1.100	ICMP	142	Echo (ping) reply id=0x9e31, seq=5/1280, ttl=62 (request in 34)
36	35.487201	192.168.2.1	224.0.0.5	OSPF	138	Hello Packet
39	40.766785	192.168.3.2	224.0.0.5	OSPF	138	Hello Packet
42	44.675591	192.168.2.1	224.0.0.5	OSPF	138	Hello Packet
45	50.185331	192.168.3.2	224.0.0.5	OSPF	138	Hello Packet

Frame 31: 142 bytes on wire (1136 bits), 142 bytes captured (1136 bits) on interface 0
 Ethernet II, Src: 0c:d6:27:2a:00:00 (0c:d6:27:2a:00:00), Dst: ca:01:2b:00:00:00
 Internet Protocol Version 4, Src: 1.1.1.1, Dst: 2.2.2.2
 Authentication Header
 Internet Protocol Version 4, Src: 192.168.1.100, Dst: 192.168.4.100
 Internet Control Message Protocol

Figure 6: ICMP traffic protected with AH in the public network.

During the ping between PC1 and PC2, a Wireshark capture was performed in the public network. The selected packet corresponds to an ICMP Echo Request protected by AH.

In the capture, the outer IP header uses the loopback addresses of the IPsec peers, with source address 1.1.1.1 and destination address 2.2.2.2. After this outer IP header, the packet contains an Authentication Header. Inside the protected packet, the original ICMP packet can still be observed, with source address 192.168.1.100 and destination address 192.168.4.100.

This confirms that the traffic between the two private hosts is transported through the IPsec tunnel. Since AH provides authentication and integrity but not encryption, the inner IP packet and the ICMP information are still visible in Wireshark.

The AH *Sequence Number* field is used as anti-replay protection. It increases for each protected packet in the Security Association, allowing the receiver to detect duplicated or replayed packets. The AH *ICV* field carries the integrity check value computed over the protected packet fields and the AH header. If the packet is modified in transit, the receiver computes a different ICV and rejects the packet. Therefore, AH protects authenticity and integrity, but it does not provide confidentiality.

OSPF packets in the public network

OSPF packets were also observed in the public network. Two different types of OSPF traffic appeared in the capture.

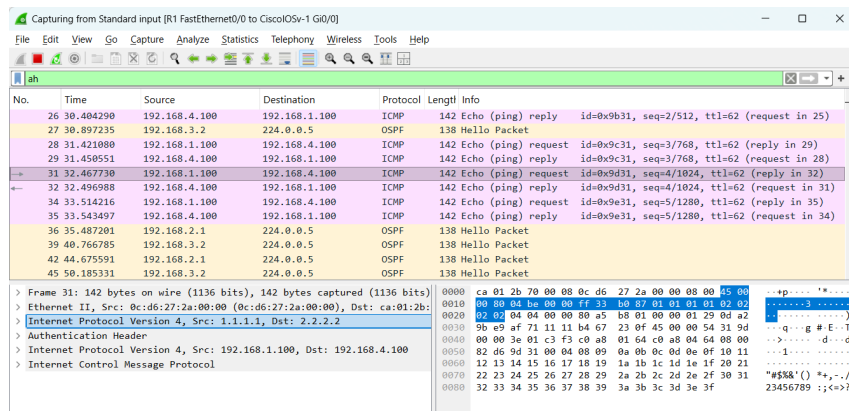


Figure 7: OSPF traffic observed in the public network.

The first type corresponds to the public OSPF process, used by the underlay network. These packets are exchanged directly between R1, RA and R2, using public addresses such as 200.1.1.1 and 200.1.1.10.

The second type corresponds to the private OSPF process of the organization. These packets appear with private source addresses, such as 192.168.2.1 and 192.168.3.2, but are transported through the IPsec tunnel. In the packet details, the outer IP header uses the IPsec tunnel endpoints, 1.1.1.1 and 2.2.2.2, and the protocol is identified as **Authentication Header** (51).

This confirms that the private OSPF traffic is protected by AH when crossing the public network.

ISAKMP exchange after clearing the crypto session

After confirming that the AH tunnel was operational, the existing crypto session was cleared in order to observe the IKEv1 negotiation again. For that purpose, the following command was executed on R1:

```
R1# clear crypto session
```

Listing 1: Command used to clear the crypto session.

After clearing the crypto session, a new ping was generated from PC1 to PC2. At the same time, a Wireshark capture was performed in the public network, between R1 and RA, using the display filter `isakmp`.

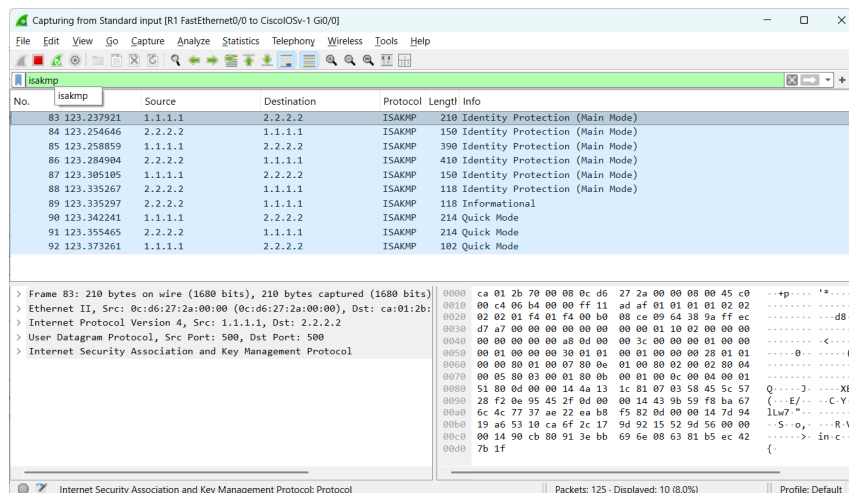


Figure 8: ISAKMP messages exchanged after clearing the crypto session.

As shown in Figure 8, the ISAKMP messages were exchanged between the two IPSec peers, 1.1.1.1 and 2.2.2.2. The first packets correspond to *Identity Protection*, also known as Main Mode. These packets are used in IKE Phase 1 to negotiate the ISAKMP Security Association between the two routers.

During Main Mode, the routers agree on the cryptographic parameters configured in the ISAKMP policy, such as AES-256 encryption, SHA hashing, Diffie-Hellman group 5 and pre-shared key authentication. This phase creates a secure ISAKMP channel that is then used to protect the next negotiation phase.

The capture also shows an *Informational* message. This type of message is used to notify the peer about changes related to existing security associations, for example when an old SA is deleted after the session is cleared.

Finally, the capture shows *Quick Mode* messages. These correspond to IKE Phase 2. In this phase, the routers negotiate the IPSec Security Associations used to protect the actual traffic between the private networks. In this case, the negotiated transform set used AH with SHA-HMAC.

In the Diffie-Hellman phase, the *Key Exchange* payload carries each peer's public Diffie-Hellman value. The value itself is not the final key; it is combined with the private Diffie-Hellman value of the local router and the value received from the peer to derive shared keying material. The nonces exchanged in the negotiation provide freshness and prevent the same keying material from being reused across different negotiations.

After the Quick Mode exchange, the IPSec tunnel was established again and the ping between PC1 and PC2 worked successfully.

Peer mismatch test

The negotiation was also analysed for the case where the peers did not agree on the IPSec parameters, for example when one side expected AH and the other side expected ESP. In that situation, IKE Phase 1 can still complete if the ISAKMP policies match, but IKE Phase 2 fails because no common IPSec transform set exists. The practical result is that `show crypto isakmp sa` may show an IKE SA, but the IPSec SA is not installed correctly and the packet counters in `show crypto ipsec sa` do not increase for protected traffic. This confirms that both peers must agree on the Phase 2 transform set, including the IPSec protocol, integrity algorithm and encryption algorithm.

ESP Security Association

After changing the transform set from AH to ESP, the crypto session was cleared and new traffic was generated between PC1 and PC2. The IPsec Security Association was checked again with the command `show crypto ipsec sa`.

```
R1#show crypto ipsec sa

interface: Tunnel0
  Crypto map tag: Tunnel0-head-0, local addr 1.1.1.1

protected vrf: (none)
local ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
remote ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
current_peer 2.2.2.2 port 500
  PERMIT, flags={origin_is_acl,}
  #pkts encaps: 17, #pkts encrypt: 17, #pkts digest: 17
  #pkts decaps: 16, #pkts decrypt: 16, #pkts verify: 16
  #pkts compressed: 0, #pkts decompressed: 0
  #pkts not compressed: 0, #pkts compr. failed: 0
  #pkts not decompressed: 0, #pkts decompress failed: 0
  #send errors 0, #recv errors 0

  local crypto endpt.: 1.1.1.1, remote crypto endpt.: 2.2.2.2
  plaintext mtu 1438, path mtu 1500, ip mtu 1500, ip mtu idb GigabitEthernet0/0
  current outbound spi: 0x28ED47E9(686639081)
  PFS (V/N): N, DH group: none

inbound esp sas:
  spi: 0xA44A07F3(2756315123)
    transform: esp-aes esp-sha-hmac ,
    in use settings ={Tunnel, }
    conn id: 11, flow_id: SW:11, sibling_flags 80000040, crypto map: Tunnel0-head-0
    sa timing: remaining key lifetime (k/sec): (4147499/3565)
    IV size: 16 bytes
    replay detection support: Y
    Status: ACTIVE(ACTIVE)

inbound ah sas:

inbound pcp sas:

outbound esp sas:
  spi: 0x28ED47E9(686639081)
    transform: esp-aes esp-sha-hmac ,
    in use settings ={Tunnel, }
    conn id: 12, flow_id: SW:12, sibling_flags 80000040, crypto map: Tunnel0-head-0
    sa timing: remaining key lifetime (k/sec): (4147499/3565)
    IV size: 16 bytes
    replay detection support: Y
    Status: ACTIVE(ACTIVE)

outbound ah sas:

outbound pcp sas:

R1#
```

Figure 9: IPsec Security Association after changing the transform set to ESP.

The output confirms that the IPsec tunnel was active using ESP. The transform set shown in the inbound and outbound SAs was `esp-aes esp-sha-hmac`. The packet counters also increased after the ping, with packets being encapsulated, encrypted and digested in the outbound direction, and decapsulated, decrypted and verified in the inbound direction.

```
#pkts encaps: 17, #pkts encrypt: 17, #pkts digest: 17
#pkts decaps: 16, #pkts decrypt: 16, #pkts verify: 16
```

Listing 2: ESP packet counters after the ping test.

This confirms that ESP was protecting the traffic between the two sites.

ESP packet capture

After changing the transform set to ESP, a new ping was performed between PC1 and PC2 while capturing traffic in the public network. The Wireshark display filter `esp` was used to show only ESP packets.

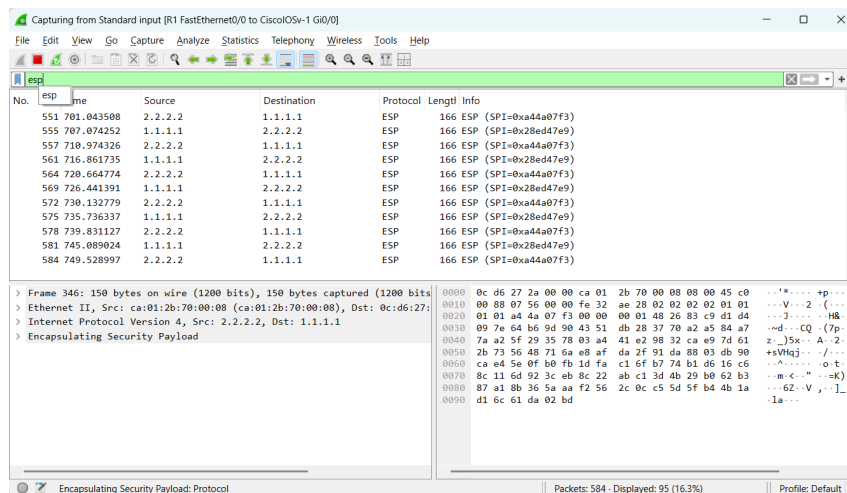


Figure 10: ESP packets captured in the public network during the ping between PC1 and PC2.

As shown in Figure 10, the packets exchanged between the IPSec peers use ESP. The outer IP header contains the tunnel endpoint addresses, 1.1.1.1 and 2.2.2.2. The packet details show the *Encapsulating Security Payload* header.

Unlike AH, the original ICMP packet is not visible in the capture. The addresses of the private hosts, 192.168.1.100 and 192.168.4.100, are also not visible. This happens because ESP encrypts the protected payload, while AH only provides authentication and integrity.

Therefore, this capture confirms that ESP provides confidentiality for the traffic exchanged between the two private sites.

Comparison between AH and ESP

The captures show a clear difference between AH and ESP. With AH, the packet was protected with an Authentication Header, but the inner IP packet was still visible in Wireshark. This allowed the original ICMP traffic between 192.168.1.100 and 192.168.4.100 to be observed.

With ESP, the public capture only showed ESP packets between 1.1.1.1 and 2.2.2.2. The original ICMP packet and the private addresses were no longer visible. This means that ESP provided confidentiality, while AH did not.

In both cases, the public network only saw traffic between the IPSec tunnel endpoints. The private traffic was transported through the tunnel.

Firewall requirements

For the IPSec tunnel to be established, the firewall must allow the IKEv1 negotiation traffic and the IPSec data traffic. IKEv1 uses UDP port 500. When AH is used, the firewall must also allow IP protocol 51. When ESP is used, the firewall must allow IP protocol 50.

Therefore, for this experiment, a firewall between R1 and R2 would need to allow UDP 500 and either AH or ESP, depending on the IPSec transform set being used.

1.1.3 IKEv1 with digital signatures

In this part of the experiment, the authentication method used by IKEv1 was changed from pre-shared keys to digital signatures. Router RA was configured as a Certification Authority, while R1 and R2 were configured as PKI clients. After obtaining their certificates through SCEP, R1 and R2 used RSA signatures to authenticate each other during the IKEv1 negotiation.

NTP and Certification Authority

Before enrolling the routers, time synchronization was required because certificate validation depends on the certificate validity interval. RA was configured as the NTP master and R1/R2 were configured to use RA as their NTP server. RA was then configured as the Certification Authority with HTTP enabled, automatic certificate granting and issuer name **cn=saarCA**.

```
RA
ntp master
ip http server
crypto pki server myCA
  issuer-name cn=saarCA
  lifetime certificate 365
  grant auto
  no shutdown

R1 and R2
ntp server 200.1.1.10
```

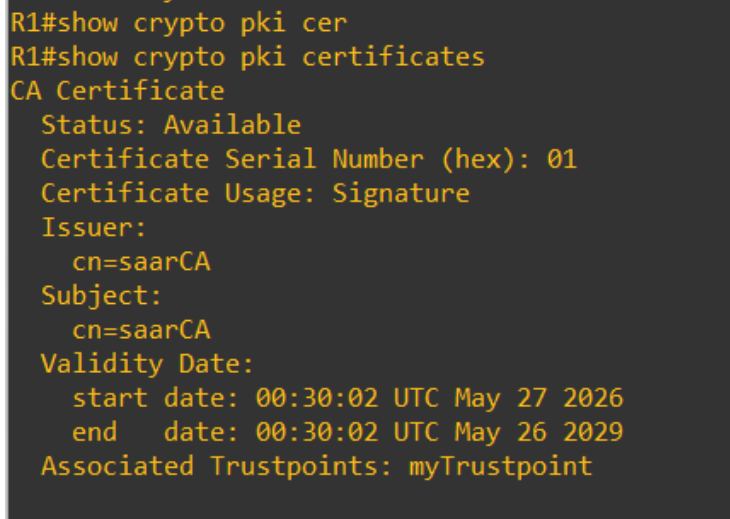
Listing 3: Commands used to prepare the CA and synchronize router time.

The CA certificate was installed in the trustpoint used by the clients.

The CA state and CA certificate were verified with:

```
show crypto pki server
show crypto ca certificate
show crypto ca certificate verbose
show crypto key mypub rsa
```

Listing 4: IOS commands used to verify the CA and CA certificate.



```
R1#show crypto pki cer
R1#show crypto pki certificates
CA Certificate
  Status: Available
  Certificate Serial Number (hex): 01
  Certificate Usage: Signature
  Issuer:
    cn=saarCA
  Subject:
    cn=saarCA
  Validity Date:
    start date: 00:30:02 UTC May 27 2026
    end   date: 00:30:02 UTC May 26 2029
  Associated Trustpoints: myTrustpoint
```

Figure 11: CA certificate associated with the trustpoint used for SCEP enrollment.

Certificate installation and public keys

The first step was to configure RA as a Certification Authority and to enroll R1 and R2 as PKI clients. The routers obtained the CA certificate and then requested their own router certificates.

The installation of the certificates was verified with the command:

```
show crypto pki certificates
```

Listing 5: Command used to verify installed PKI certificates.

```
R1#show crypto pki certificates
Certificate
  Status: Available
  Certificate Serial Number (hex): 02
  Certificate Usage: General Purpose
  Issuer:
    cn=saarCA
  Subject:
    Name: R1.saar.local
    Serial Number: 9BGWZKFU71NSTYUMX0TFY
    hostname=R1.saar.local+serialNumber=9BGWZKFU71NSTYUMX0TFY
    cn=R1
  Validity Date:
    start date: 00:38:07 UTC May 27 2026
    end   date: 00:38:07 UTC May 27 2027
  Associated Trustpoints: myTrustpoint

CA Certificate
  Status: Available
  Certificate Serial Number (hex): 01
  Certificate Usage: Signature
  Issuer:
    cn=saarCA
  Subject:
    cn=saarCA
  Validity Date:
    start date: 00:30:02 UTC May 27 2026
    end   date: 00:30:02 UTC May 26 2029
  Associated Trustpoints: myTrustpoint

R1#
```

Figure 12: Certificates installed on R1.

```

R2#show crypto pki certificates
Certificate
  Status: Available
  Certificate Serial Number (hex): 03
  Certificate Usage: General Purpose
  Issuer:
    cn=saarCA
  Subject:
    Name: R2.saar.local
    hostname=R2.saar.local
    cn=R2
  Validity Date:
    start date: 00:42:53 UTC May 27 2026
    end   date: 00:42:53 UTC May 27 2027
  Associated Trustpoints: myTrustpoint

CA Certificate
  Status: Available
  Certificate Serial Number (hex): 01
  Certificate Usage: Signature
  Issuer:
    cn=saarCA
  Subject:
    cn=saarCA
  --More--
May 27 00:43:04.881: %SYS-5-CONFIG_I: Configured from console by console
  Validity Date:
    start date: 00:30:02 UTC May 27 2026
    end   date: 00:30:02 UTC May 26 2029
  Associated Trustpoints: myTrustpoint

```

Figure 13: Certificates installed on R2.

The public keys of R1 and R2 were also checked using:

```
show crypto key mypub rsa
```

Listing 6: Command used to display the local RSA public keys.

```

R1#show crypto key mypub rsa
% Key pair was generated at: 00:31:32 UTC May 27 2026
Key name: R1
Key type: RSA KEYS
Storage Device: private-config
Usage: General Purpose Key
Key is not exportable.
Key Data:
 30819F30 0D06092A 864886F7 0D010101 05000381 8D003081 89028181 00A809C8
 4D6E5AF5 19A664AE 1F33A9E5 5FE593D2 1EE67A24 885D3DD2 0A39240D 1373408E
 28814D92 9461B53C 76801565 8BE3D56A FC0B7DE7 879D1BF3 C415ECEB 2648AE46
 028D5A5F 8E00B94C D388FD7B D44456E9 CC5FE53F 3F755524 27389C0B CECE8424
 F90CD273 36A3A700 A8AF3659 1C63F84A EC72437B 01B6C0A1 5E713A47 1F020301
 0001
% Key pair was generated at: 00:31:32 UTC May 27 2026
Key name: R1.server
Key type: RSA KEYS
Temporary key
Usage: Encryption Key
Key is not exportable.
Key Data:
 307C300D 06092A86 4886F70D 01010105 00036B00 30680261 00B54D8D B9202E82
 609E92D4 CECC0877 A72C077C E009AEDD 647DDE08 B4EBF976 8DCCCBAA 3D1132BE
 4F122538 C97062AD C07665D1 96A77B92 4A14ABF8 E2A378BC 167EC463 869AC9C0
 4E881AC2 9746E68E B4CD699B B577C806 D9751890 F01CEDD6 63020301 0001
R1#

```

Figure 14: Public keys generated on R1.

```

R2#show crypto key mypub rsa
% Key pair was generated at: 00:42:09 UTC May 27 2026
Key name: R2
Key type: RSA KEYS
Storage Device: private-config
Usage: General Purpose Key
Key is not exportable.
Key Data:
30819F30 0D06092A 864886F7 0D010101 05000381 8D003081 89028181 00B86D77
34735D4E 702BD352 C2C84158 CC81D0D1 57430ED4 3CBD346F C73CF217 8F1985E9
1D107F5D 7BB33AD9 73592560 44AA925C A88DAD60 BA9CE57D 8C504A5B BF93913A
0D54621E 7468E7B3 D25D099C 46424E6B 4BB67C4D 651557A8 C18048C5 6D4F3314
3D100C31 74CC16DE 1763D3DA EE4011DD 009634D5 41DF3A49 D9ED09DE 3F020301
0001
% Key pair was generated at: 00:42:09 UTC May 27 2026
Key name: R2.server
Key type: RSA KEYS
Temporary key
Usage: Encryption Key
Key is not exportable.
Key Data:
307C300D 06092A86 4886F70D 01010105 00036B00 30680261 00B8FBA1 E93A52FC
A980DBC2 47C6A289 8CF4B477 BF9B1B9A 90CC8E33 5CECD5B9 493FDD08 2A6439AB
DE1154A7 C59578CA ACB2E125 42AD34A4 A16FB53C F0E72455 8F82ADD6 ADE4F8F5
C2159684 695D87AC E007398A 2F65844A 6CA10D52 BBE250A0 73020301 0001
R2#

```

Figure 15: Public keys generated on R2.

These outputs confirm that both routers had valid certificates and RSA key pairs installed.

SCEP-based enrollment

The enrollment process was performed using SCEP. During this process, R1 and R2 contacted the CA through HTTP in order to obtain the CA certificate and request their own certificates.

A Wireshark capture was performed during the enrollment process using the filter `http`.

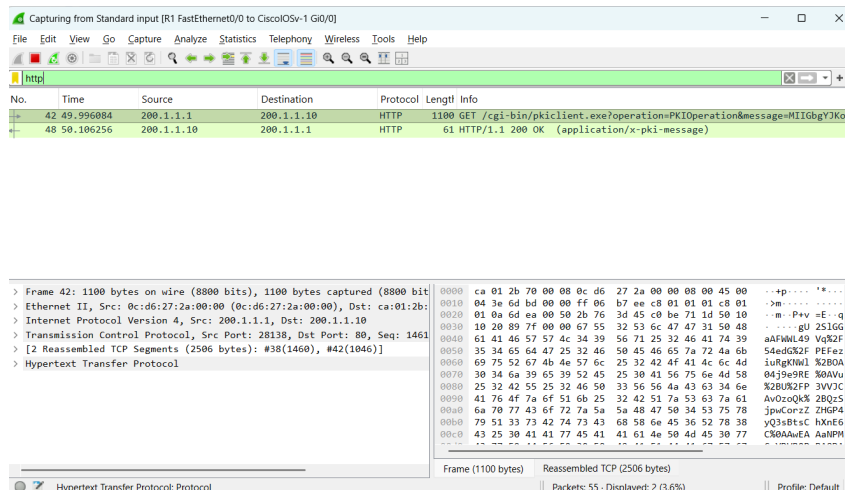


Figure 16: HTTP/SCEP messages exchanged during certificate enrollment.

As shown in Figure 16, R1 sent an HTTP GET request to the CA at 200.1.1.10. The request used the SCEP endpoint `/cgi-bin/pkiclient.exe` with the operation `PKIOperation`. The CA replied with `HTTP/1.1 200 OK` and the content type `application/x-pki-message`.

These SCEP messages are used to exchange certificate-related information between the router and the CA. The first SCEP exchanges obtain the CA certificate, allowing the router to trust certificates signed by `saarCA`. The following `PKIOperation` messages transport the certificate

request and the issued router certificate. In this case, R1 used SCEP to request and receive its router certificate signed by the CA. The certificate can be exported from the HTTP payload in Wireshark and decoded separately, because the response carries a certificate object inside the application/x-pki-message body.

ISAKMP negotiation with digital signatures

After the certificates were installed, the ISAKMP policy was changed to use RSA signatures:

```
crypto isakmp policy 10
 hash sha
 authentication rsa-sig
 group 5
 lifetime 86400
 encryption aes 256
```

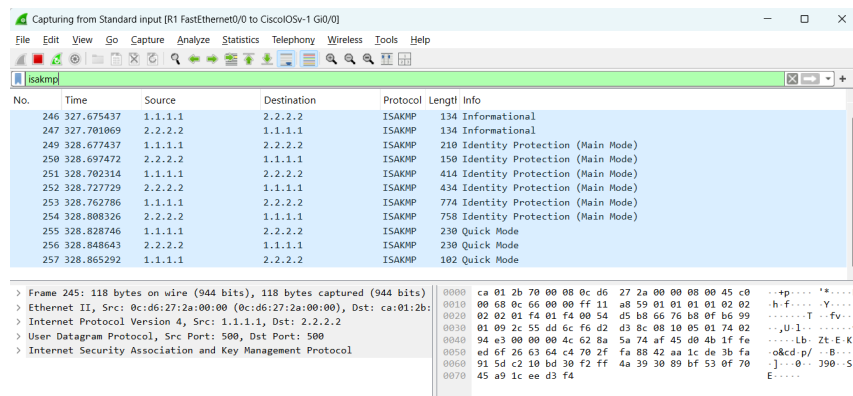
Listing 7: ISAKMP policy configured for RSA signature authentication.

The pre-shared key configuration was removed. The IPSec tunnel was then configured with ESP:

```
crypto ipsec transform-set myTSet esp-aes esp-sha-hmac
```

Listing 8: IPSec transform set configured with ESP.

To observe the IKEv1 negotiation, the crypto session was cleared and a new ping was generated between PC1 and PC2. A Wireshark capture was performed using the filter `isakmp`.



No.	Time	Source	Destination	Protocol	Length	Info
246	327.675437	1.1.1.1	2.2.2.2	ISAKMP	134	Informational
247	327.701069	2.2.2.2	1.1.1.1	ISAKMP	134	Informational
249	328.677437	1.1.1.1	2.2.2.2	ISAKMP	210	Identity Protection (Main Mode)
250	328.697472	2.2.2.2	1.1.1.1	ISAKMP	150	Identity Protection (Main Mode)
251	328.702314	1.1.1.1	2.2.2.2	ISAKMP	414	Identity Protection (Main Mode)
252	328.727729	2.2.2.2	1.1.1.1	ISAKMP	434	Identity Protection (Main Mode)
253	328.762786	1.1.1.1	2.2.2.2	ISAKMP	774	Identity Protection (Main Mode)
254	328.808326	2.2.2.2	1.1.1.1	ISAKMP	758	Identity Protection (Main Mode)
255	328.828746	1.1.1.1	2.2.2.2	ISAKMP	230	Quick Mode
256	328.848643	2.2.2.2	1.1.1.1	ISAKMP	230	Quick Mode
257	328.865292	1.1.1.1	2.2.2.2	ISAKMP	102	Quick Mode

Figure 17: ISAKMP messages exchanged when using digital signatures.

Compared with the pre-shared key configuration, the main difference is the authentication method. With pre-shared keys, both routers authenticate using the same shared secret. With digital signatures, each router uses its own certificate and private key, while the peer validates the certificate using the CA certificate. Therefore, authentication depends on the PKI chain of trust instead of on a manually configured shared secret. During the ISAKMP exchange, the peers use their identities, certificates and RSA signatures to prove possession of the corresponding private keys. If the certificate cannot be validated against the CA, the RSA-signature authentication step fails and the IKEv1 negotiation cannot complete successfully.

Encrypted ICMP traffic

After the ISAKMP negotiation, a ping was performed from PC1 to PC2. At the same time, traffic was captured in the public network using the Wireshark filter `esp`.

No.	Time	Source	Destination	Protocol	Length	Info
225	300.632933	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0x28811619)
226	305.656469	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x42819b7d)
231	310.236571	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0x28811619)
234	314.963789	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x42819b7d)
238	319.538795	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0x28811619)
242	324.507131	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x42819b7d)
258	328.866349	1.1.1.1	2.2.2.2	ESP	150	ESP (SPI=0xbf86735f)
259	328.888954	2.2.2.2	1.1.1.1	ESP	150	ESP (SPI=0xbf86735f)
260	328.891602	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0xbf86735f)
261	328.909127	2.2.2.2	1.1.1.1	ESP	150	ESP (SPI=0xbf86735f)
262	328.909154	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0xbf86735f)
263	328.913449	1.1.1.1	2.2.2.2	ESP	150	ESP (SPI=0xbf86735f)

Figure 18: ESP packets captured during the ping between PC1 and PC2.

The capture shows ESP packets between the IPsec endpoints 1.1.1.1 and 2.2.2.2. The original ICMP packets exchanged between PC1 and PC2 are not visible in the public network. This confirms that the ICMP traffic was encrypted by ESP.

Public key of the peer after ISAKMP exchange

After the ISAKMP exchange, each router stored information about the public key of the peer. This was verified with IOS show commands.

```
show crypto key pubkey-chain rsa
```

Listing 9: Command used to inspect the RSA public key chain.

```

R1# show crypto key pubkey-chain rsa
Codes: M - Manually configured, C - Extracted from certificate

Code Usage      IP-Address/VRF      Keyring      Name
C   Signing
C   Signing      default             cn=Cisco Root CA M1,o=Cisco
C   Signing      default             cn=Cisco Root CA 2048,o=Cisco Systems
C   Signing      default             cn=Cisco Manufacturing CA,o=Cisco Systems
C   Signing      default             ou=Class 3 Public Primary Certification Authority,o=VeriSign, Inc.,c=US
C   Signing      default             cn=Cisco Root CA R2,o=Cisco
C   Signing      default             cn=Cisco Manufacturing CA SHA2,o=Cisco
C   Signing      default             cn=Licensing Root - DEV,o=Cisco
C   Signing      default             cn=Cisco Licensing Root CA,o=Cisco
C   Signing      default             cn=Cisco RXC-R2,o=Cisco Systems,c=US
C   Signing      default             cn=saarCA
C   Signing      default             R2.saar.local

```

Figure 19: Public key of the peer stored after the ISAKMP exchange.

As shown in Figure 19, R1 has an entry associated with R2.saar.local. The code C indicates that the key was extracted from a certificate. This confirms that, during the certificate-based ISAKMP negotiation, R1 obtained and stored the public key information required to validate R2's digital signature.

Certificate revocation

Finally, the certificates issued to R1 and R2 were revoked at the Certification Authority. The serial number of the R1 certificate was 02, while the serial number of the R2 certificate was 03. The certificates were revoked on the CA using the following commands:

```

RA# crypto pki server myCA revoke 02
% Certificate 02 successfully revoked.

RA# crypto pki server myCA revoke 03
% Certificate 03 successfully revoked.

```

Listing 10: Commands used to revoke the router certificates on the CA.

After revoking both certificates, the crypto sessions were cleared and a new ping was performed from PC1 to PC2. The communication was no longer successful:

```
PC1> ping 192.168.4.100

*192.168.1.3 icmp_seq=1 ttl=255 time=9.834 ms
(ICMP type:3, code:1, Destination host unreachable)

*192.168.1.3 icmp_seq=2 ttl=255 time=5.116 ms
(ICMP type:3, code:1, Destination host unreachable)

*192.168.1.3 icmp_seq=3 ttl=255 time=5.312 ms
(ICMP type:3, code:1, Destination host unreachable)

*192.168.1.3 icmp_seq=4 ttl=255 time=5.506 ms
(ICMP type:3, code:1, Destination host unreachable)

*192.168.1.3 icmp_seq=5 ttl=255 time=6.487 ms
(ICMP type:3, code:1, Destination host unreachable)
```

Listing 11: Ping output after certificate revocation.

The ping failed after the certificate revocation because R1 and R2 could no longer authenticate each other during the IKEv1 negotiation. After the restart and the session clear, the routers used the CA trustpoint and the SCEP/HTTP-based PKI information to validate the peer certificates, including revocation information. Since the certificates used by R1 and R2 had been revoked on the CA, certificate validation failed during RSA-signature authentication. As a result, IKEv1 Phase 1 could not complete, no valid ISAKMP/IPSec SAs were installed, and the tunnel could not carry the ICMP traffic.

The message `Destination host unreachable` was generated by 192.168.1.3, which is the default gateway of PC1. This indicates that the packet could not be forwarded through the private network towards PC2, because the protected tunnel between the two sites was no longer available.

1.2 IPsec with IKEv2

1.2.1 IKEv2 configuration

Following the IKEv2 part of the lab guide [1], the IPsec tunnel between R1 and R2 was reconfigured to use IKEv2 instead of IKEv1. The same topology and addressing plan from the previous sections were kept. The tunnel endpoints remained the loopback interfaces of R1 and R2, with addresses 1.1.1.1 and 2.2.2.2.

Authentication was performed with a pre-shared key, and the IPsec tunnel used ESP with AES encryption and SHA-HMAC authentication. The IKEv2 proposal was configured with AES-CBC-256 encryption, SHA-512 integrity, SHA-512 PRF and Diffie-Hellman group 16.

The main IKEv2-related configuration used in this experiment is summarized below:

```
crypto ikev2 proposal myProposal
  encryption aes-cbc-256
  integrity sha512
  prf sha512
  group 16
```

```

crypto ikev2 policy myIKEv2Policy
proposal myProposal

crypto ikev2 keyring myKeyring
peer <peer-name>
address <peer-loopback>
pre-shared-key saar

crypto ikev2 profile myIKEv2Profile
match identity remote address <peer-loopback> 255.255.255.255
authentication local pre-share
authentication remote pre-share
keyring local myKeyring

crypto ipsec transform-set myTSet esp-aes esp-sha-hmac

crypto ipsec profile myIPSecProfile
set transform-set myTSet
set ikev2-profile myIKEv2Profile

```

Listing 12: IKEv2 and IPsec configuration used for the tunnel.

After applying the configuration on both routers, connectivity between the two private sites was tested from PC1 to PC2 and the ping was successful. This confirmed that the tunnel had been established correctly.

The IKEv2 and IPsec state was checked with the IOS commands requested in the lab guide:

```

show crypto ikev2 proposal
show crypto ikev2 policy
show crypto ikev2 sa detailed
show crypto ipsec profile
show crypto ipsec sa

```

Listing 13: IOS show commands used to verify the IKEv2/IPsec configuration.

1.2.2 IKEv2 Security Association

The IKEv2 Security Association was verified using the command:

```

show crypto ikev2 sa detailed

```

Listing 14: Command used to verify the detailed IKEv2 Security Association.

```

R1#
R1#show crypto ikev2 sa detailed
IPv4 Crypto IKEv2 SA

Tunnel-id Local Remote fvr/fivr Status
1 1.1.1.1/500 2.2.2.2/500 none/none READY
Encr: AES-CBC, keysize: 256, PRF: SHA512, Hash: SHA512, DH Grp:16, Auth sign: PSK, Auth verif
y: PSK
Life/Active Time: 86400/304 sec
CE id: 1022, Session-id: 3
Status Description: Negotiation done
Local spi: A056909C50DD8071 Remote spi: ACE435643F955C76
Local id: 1.1.1.1
Remote id: 2.2.2.2
Local req msg id: 0 Remote req msg id: 2
Local next msg id: 0 Remote next msg id: 2
Local req queued: 0 Remote req queued: 2
Local window: 5 Remote window: 5
DPD configured for 0 seconds, retry 0
Fragmentation not configured.
Dynamic Route Update: disabled
Extended Authentication not configured.
NAT-T is not detected
Cisco Trust Security SGT is disabled
Initiator of SA : No

IPv6 Crypto IKEv2 SA

```

Figure 20: Detailed IKEv2 Security Association information.

As shown in Figure 20, the IKEv2 Security Association between R1 and R2 was successfully established. The status is **READY** and the status description indicates that the negotiation was completed.

The output also confirms the negotiated parameters: AES-CBC with a 256-bit key for encryption, SHA-512 for integrity and PRF, Diffie-Hellman group 16, and pre-shared key authentication. The local and remote SPIs are also shown, identifying the IKEv2 Security Association created between the two peers.

1.2.3 IPsec Security Association

The IPsec tunnel status was verified with the command:

```
show crypto ipsec sa
```

Listing 15: Command used to verify the IPsec Security Association.

```

R1#show crypto ipsec sa
interface: Tunnel0
  Crypto map tag: Tunnel0-head-0, local addr 1.1.1.1

protected vrf: (none)
local ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
remote ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
current_peer 2.2.2.2 port 500
  PERMIT, flags={origin_is_acl,}
  #pkts encaps: 65, #pkts encrypt: 65, #pkts digest: 65
  #pkts decaps: 63, #pkts decrypt: 63, #pkts verify: 63
  #pkts compressed: 0, #pkts decompressed: 0
  #pkts not compressed: 0, #pkts compr. failed: 0
  #pkts not decompressed: 0, #pkts decompress failed: 0
  #send errors 0, #recv errors 0

  local crypto endpt.: 1.1.1.1, remote crypto endpt.: 2.2.2.2
  plaintext mtu 1438, path mtu 1500, ip mtu 1500, ip mtu idb GigabitEthernet0/0
  current outbound spi: 0x5C1E2889(1545480329)
  PFS (Y/N): N, DH group: none

inbound esp sas:
  spi: 0x16C1B1B(23862043)
    transform: esp-aes esp-sha-hmac ,
    in use settings = {Tunnel, }
    conn id: 1, flow id: SW:1, sibling_flags 80000040, crypto map: Tunnel0-head-0
    sa timing: remaining key lifetime (k/sec): (4151850/3109)
    IV size: 16 bytes
    replay detection support: Y
    Status: ACTIVE(ACTIVE)

inbound ah sas:

inbound pcp sas:

outbound esp sas:
  spi: 0x5C1E2889(1545480329)
    transform: esp-aes esp-sha-hmac ,
    in use settings = {Tunnel, }
    conn id: 2, flow id: SW:2, sibling_flags 80000040, crypto map: Tunnel0-head-0
    sa timing: remaining key lifetime (k/sec): (4151850/3109)
    IV size: 16 bytes
    replay detection support: Y
    Status: ACTIVE(ACTIVE)

outbound ah sas:

outbound pcp sas:
R1#

```

Figure 21: IPSec Security Association established using IKEv2.

As shown in Figure 21, the IPSec Security Association was active on **Tunnel0**. The local crypto endpoint was 1.1.1.1 and the remote crypto endpoint was 2.2.2.2.

The output also shows that the inbound and outbound ESP SAs were active, both using the transform set **esp-aes esp-sha-hmac**. The packet counters increased during the ping test, with packets being encapsulated, encrypted and digested in the outbound direction, and decapsulated, decrypted and verified in the inbound direction. This confirms that the traffic between the two private sites was protected by IPSec.

1.2.4 Wireshark analysis of the IKEv2 exchange

To observe the IKEv2 negotiation, a Wireshark capture was performed in the public network while the crypto session was cleared and new traffic was generated. The display filter **ikev2** was used.

No.	Time	Source	Destination	Protocol	Length	Info
41	57.073682	1.1.1.1	2.2.2.2	ISAKMP	138	INFORMATIONAL MID=00 Responder Request
42	57.074483	1.1.1.1	2.2.2.2	ISAKMP	154	INFORMATIONAL MID=01 Responder Request
43	57.088608	2.2.2.2	1.1.1.1	ISAKMP	138	INFORMATIONAL MID=00 Initiator Response
44	57.098783	2.2.2.2	1.1.1.1	ISAKMP	138	INFORMATIONAL MID=01 Initiator Response
45	57.955387	1.1.1.1	2.2.2.2	ISAKMP	816	IKE_SA_INIT MID=00 Initiator Request
46	58.044639	2.2.2.2	1.1.1.1	ISAKMP	849	IKE_SA_INIT MID=00 Responder Response
47	58.119820	1.1.1.1	2.2.2.2	ISAKMP	666	IKE_AUTH MID=01 Initiator Request
48	58.145345	2.2.2.2	1.1.1.1	ISAKMP	362	IKE_AUTH MID=01 Responder Response

<pre> > Frame 46: 849 bytes on wire (6792 bits), 849 bytes captured (6792 bits) > Ethernet II, Src: ca:01:2b:70:00:08 (ca:01:2b:70:00:08), Dst: 0c:d6:27: > Internet Protocol Version 4, Src: 2.2.2.2, Dst: 1.1.1.1 > User Datagram Protocol, Src Port: 500, Dst Port: 500 > Internet Security Association and Key Management Protocol </pre>	<pre> 0000 0c d6 27 2a 00 00 ca 01 2b 70 00 08 08 00 45 c0 ..*....+p... 0010 03 43 02 98 00 00 fe 11 b0 4c 02 02 02 01 01 :C.....L... 0020 01 01 01 f4 01 f4 03 2f 05 9f e1 76 d6 d3 77 85 /...v... 0030 b1 df 6d e0 88 ed da 9f 16 62 21 20 22 20 00 00 ...m....bl" 0040 00 00 00 00 03 27 22 00 00 30 00 00 00 2c 01 01 0... 0050 00 04 03 00 00 0c 01 00 00 0c 00 0e 01 00 03 00 0060 00 08 02 00 00 07 03 00 00 08 03 00 00 0e 00 00 </pre>
--	---

Figure 22: IKEv2 messages captured in the public network.

As shown in Figure 22, the IKEv2 negotiation was performed between 1.1.1.1 and 2.2.2.2. The capture shows the `IKE_SA_INIT` exchange, followed by the `IKE_AUTH` exchange.

The `IKE_SA_INIT` messages are used to create the initial IKE Security Association. In this exchange, the routers negotiate the cryptographic proposal, exchange Diffie-Hellman public values and send nonces used to generate fresh keying material.

The most important fields in `IKE_SA_INIT` are the initiator and responder SPIs, the SA payload, the Key Exchange payload and the Nonce payload. The SPIs identify the IKEv2 SA. The SA payload carries the cryptographic proposals, including encryption, integrity, PRF and Diffie-Hellman group. The Key Exchange payload carries the public Diffie-Hellman value of each peer. The nonce is a fresh random value used in key derivation, preventing old keying material from being reused.

The `IKE_AUTH` messages are then used to authenticate the peers and complete the setup of the IPSec tunnel. In this experiment, the peers authenticated each other using the pre-shared key `saar`. These messages also complete the negotiation of the IPSec Security Associations used by ESP. Unlike the first `IKE_SA_INIT` exchange, the sensitive contents of `IKE_AUTH` are protected inside the encrypted payload of the IKEv2 SA. This is why Wireshark can identify the message type but cannot decode the authenticated identities and traffic selectors without the negotiated keys.

Compared with IKEv1, the IKEv2 exchange is more compact and uses fewer message types. Instead of the Main Mode and Quick Mode exchanges observed in IKEv1, IKEv2 uses a simpler sequence based mainly on `IKE_SA_INIT` and `IKE_AUTH`. In this experiment, these messages were sufficient to authenticate both peers and establish the IPSec tunnel.

1.2.5 Encrypted traffic with ESP

After the tunnel was established, a Wireshark capture was performed in the public network during a ping from PC1 to PC2. The display filter `esp` was used.

Capturing from Standard input [R1 FastEthernet0/0 to Cisco/OSv-1 Gi0/0]

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

esp

No.	Time	Source	Destination	Protocol	Length	Info
215	270.974153	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0xc64e1a5a)
216	271.990765	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x75abe8ef)
217	272.022290	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0xc64e1a5a)
220	273.038694	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x75abe8ef)
221	273.059782	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0xc64e1a5a)
222	274.076670	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x75abe8ef)
223	274.106101	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0xc64e1a5a)
224	275.124717	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x75abe8ef)
225	275.152406	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0xc64e1a5a)
226	276.139433	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0xc64e1a5a)
227	277.261024	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x75abe8ef)

> Frame 220: 166 bytes on wire (1328 bits), 166 bytes captured (1328 bits)		0000	ca 01 2b 70 00 08 0c d6 27 2a 00 00 08 00 45 00	...	...
> Ethernet II, Src: 0c:d6:27:2a:00:00 (0c:d6:27:2a:00:00), Dst: ca:01:2b:70:00:08 (ca:01:2b:70:00:08)		0010	00 58 02 de 00 00 ff 32 b2 50 01 01 01 01 02 02	...	...
> Internet Protocol Version 4, Src: 1.1.1.1, Dst: 2.2.2.2		0020	02 02 75 ab e8 ef 00 00 00 23 c3 d8 fc c0 69 04	...	...
> Encapsulating Security Payload		0030	38 02 bf 0b 7b a6 8c 19 b4 97 57 28 6c 9f 1c 75	...	...
		0040	44 9a 3a 99 98 90 9f 1e f6 73 13 50 91 9e 37 dd	...	...
		0050	66 42 92 27 f8 ec 0e 59 06 6e 9b 0f 7b 12 e3 2f	...	...

Figure 23: ESP packets captured in the public network during the ping between PC1 and PC2.

As shown in Figure 23, the packets observed in the public network are ESP packets exchanged between the IPsec tunnel endpoints 1.1.1.1 and 2.2.2.2. The original ICMP packets exchanged between PC1 and PC2 are not visible in the capture.

This confirms that ESP encrypts the protected traffic before it crosses the public network.

1.2.6 IKEv2 SA rekeying

To observe the IKEv2 SA rekeying process, the lifetime of the IKEv2 SA was reduced to 180 seconds on R2:

```
crypto ikev2 profile myIKEv2Profile
lifetime 180
```

Listing 16: IKEv2 profile lifetime configured for rekeying.

After saving the configuration, the crypto session was cleared and new traffic was generated between PC1 and PC2. During this process, a Wireshark capture was performed in the public network using the filter `udp.port == 500`.

Capturing from Standard input (R1 FastEthernet0/0 to CiscoG1-15v1/0)						
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
ludp.port == 500						
No.	Time	Source	Destination	Protocol	Length	Info
410	537.590113	2.2.2.2	1.1.1.1	ISAKMP	154	INFORMATIONAL MID=01 Responder Request
411	537.595407	1.1.1.1	2.2.2.2	ISAKMP	138	INFORMATIONAL MID=00 Initiator Response
412	537.598984	1.1.1.1	2.2.2.2	ISAKMP	138	INFORMATIONAL MID=01 Initiator Response
414	538.416657	2.2.2.2	1.1.1.1	ISAKMP	816	IKE_SA_INIT MID=00 Initiator Request
415	538.491785	1.1.1.1	2.2.2.2	ISAKMP	849	IKE_SA_INIT MID=00 Responder Response
416	538.578072	2.2.2.2	1.1.1.1	ISAKMP	666	IKE_AUTH MID=01 Initiator Request
417	538.597501	1.1.1.1	2.2.2.2	ISAKMP	362	IKE_AUTH MID=01 Responder Response
521	660.608039	2.2.2.2	1.1.1.1	ISAKMP	762	CREATE_CHILD_SA MID=02 Initiator Request
522	660.673476	1.1.1.1	2.2.2.2	ISAKMP	762	CREATE_CHILD_SA MID=02 Responder Response
523	660.751097	2.2.2.2	1.1.1.1	ISAKMP	138	INFORMATIONAL MID=03 Initiator Request
524	660.755359	1.1.1.1	2.2.2.2	ISAKMP	138	INFORMATIONAL MID=03 Responder Response

Frame 521: 762 bytes on wire (6096 bits), 762 bytes captured (6096 bits)

Ethernet II, Src: ca:01:2b:70:00:08 (ca:01:2b:70:00:08), Dst: 0c:d6:27:2a:00:00 (0c:d6:27:2a:00:00)

Internet Protocol Version 4, Src: 2.2.2.2, Dst: 1.1.1.1

User Datagram Protocol, Src Port: 500, Dst Port: 500

Internet Security Association and Key Management Protocol

0000 0c d6 27 2a 00 00 ca 01 2b 70 00 08 08 00 45 c0

0010 02 ec 03 d4 00 00 fe 11 af 67 02 02 02 01 01

0020 01 01 01 f4 01 f4 d2 d8 49 6c d6 9c 23 22 b8 60

0030 8f 2a bb 0f eb df a0 5f da b5 2e 20 24 08 00 00

0040 00 02 00 00 02 d0 21 00 02 b4 b2 01 86 75 85 b5

0050 03 0e 09 70 72 31 a7 12 72 ff 10 20 03 7a 45 9f

Figure 24: IKEv2 messages exchanged during IKEv2 SA rekeying.

As shown in Figure 24, the initial tunnel establishment used the IKE_SA_INIT and IKE_AUTH exchanges. Later, when the SA lifetime was close to expiring, the rekeying process generated CREATE_CHILD_SA messages.

The capture shows that the rekeying process did not start again with a new IKE_SA_INIT exchange. Instead, the existing IKEv2 SA was used to protect the exchange that created the new SA. This confirms that IKEv2 can renew the SA without repeating the full initial negotiation.

Table 2: SPI and lifetime fields used to distinguish IKEv2 and Child SA rekeying.

Moment	IKE local SPI	IKE remote SPI	ESP inbound SPI	ESP outbound SPI
Before rekey	A056909C50DD8071	ACE435643F955C76	0x16C1B1B	0x5C1E2889
After IKE SA rekey	Changed with the new IKE SA	Changed with the new IKE SA	Not the main indicator	Not the main indicator
After IPSec/Child SA rekey	Unchanged unless the IKE SA is rekeyed	Unchanged unless the IKE SA is rekeyed	Replaced by a new ESP SPI	Replaced by a new ESP SPI

The first row uses the concrete values visible in the IOS outputs before rekeying: `show crypto ikev2 sa detailed` shows the IKE SPIs, and `show crypto ipsec sa` shows the ESP inbound and outbound SPIs. During IKE SA rekeying, the IKE SPIs are the values that change. During IPSec/Child SA rekeying, the existing IKEv2 SA remains in use and the ESP SPIs are replaced instead.

1.2.7 IPSec SA rekeying

The IPSec SA rekeying test was performed by reducing the lifetime of the IPSec SA on R1:

```
crypto ipsec profile myIPSecProfile
set security-association lifetime seconds 120
```

Listing 17: IPSec SA lifetime configured for Child SA rekeying.

After saving the configuration and restarting R1, traffic was generated again between PC1 and PC2. The rekeying of the IPSec SA also uses `CREATE_CHILD_SA` messages, but its purpose is different from IKE SA rekeying. In this case, the IKEv2 SA remains active and is used to protect the exchange that creates new ESP Child SAs. The relevant values that change are the inbound and outbound ESP SPIs shown by `show crypto ipsec sa`. The IKEv2 SPIs remain associated with the existing IKE SA unless the IKE SA itself is rekeyed.

Therefore, neither IKEv2 SA rekeying nor IPSec SA rekeying restarts from scratch with a new `IKE_SA_INIT` exchange. Both use `CREATE_CHILD_SA`, but IKE SA rekeying creates a new IKE SA and changes the IKE SPIs, while IPSec SA rekeying creates new ESP SAs and changes the ESP SPIs.

1.2.8 Comparison with AI explanation

An AI tool was asked to explain the role of `IKE_SA_INIT`, `IKE_AUTH`, IKEv2 SA rekeying and IPSec SA rekeying. The explanation predicted that `IKE_SA_INIT` would negotiate algorithms and exchange Diffie-Hellman material, while `IKE_AUTH` would authenticate the peers and establish the first Child SA. It also predicted that rekeying would use `CREATE_CHILD_SA` rather than repeating the complete initial exchange.

The experimental behaviour matched that explanation. During the initial establishment, Wireshark showed the expected IKEv2 initialization and authentication exchanges. Later, it showed `CREATE_CHILD_SA` messages during rekeying. The IOS outputs confirmed the negotiated IKEv2 parameters and the active ESP Security Associations.

1.2.9 Conclusion

The experiment confirmed that the IPSec tunnel could be successfully established using IKEv2. The IOS show commands confirmed the correct establishment of both the IKEv2 and IPSec Security Associations, while the Wireshark captures showed the IKEv2 negotiation messages and the encrypted ESP traffic exchanged between the peers.

2 DMVPN Phase 3

2.1 Experimental setup

Following the DMVPN Phase 3 exercise specification [1], a topology was configured with one hub router and two spoke routers. Router R1 was configured as the hub, while R2 and R3 were configured as spokes. The public network was used only to provide underlay connectivity between the loopback interfaces of the routers.

The DMVPN overlay network used the address range 10.10.10.0/24. Router R1 used the tunnel address 10.10.10.1, R2 used 10.10.10.2, and R3 used 10.10.10.3. RIPv2 was used in the private overlay network, while OSPF was kept in the public underlay network.

Table 3: Addressing plan used in the DMVPN Phase 3 simulation.

Device	Interface	Address	Role
R1	G0/0	200.1.1.1/24	Public network, hub
R1	G0/1	192.168.1.1/24	Private network, site 1
R1	Lo0	1.1.1.1/32	Hub NBMA address
R1	Tunnel0	10.10.10.1/24	DMVPN hub
RA	F0/0	200.1.1.10/24	Public transit network
RA	F0/1	200.2.2.10/24	Public transit network
RA	F1/0	200.3.3.10/24	Public transit network
R2	G0/0	200.2.2.2/24	Public network, spoke 1
R2	G0/1	192.168.2.2/24	Private network, site 2
R2	Lo0	2.2.2.2/32	Spoke 1 NBMA address
R2	Tunnel0	10.10.10.2/24	DMVPN spoke 1
R3	G0/0	200.3.3.3/24	Public network, spoke 2
R3	G0/1	192.168.3.3/24	Private network, site 3
R3	Lo0	3.3.3.3/32	Spoke 2 NBMA address
R3	Tunnel0	10.10.10.3/24	DMVPN spoke 2
PC1	Eth0	192.168.1.100/24	End host, site 1
PC2	Eth0	192.168.2.100/24	End host, site 2
PC3	Eth0	192.168.3.100/24	End host, site 3

2.2 DMVPN Phase 3 configuration

The hub router R1 was configured with a multipoint GRE tunnel, NHRP redirect support and RIP route summarization:

```
interface Tunnel0
ip address 10.10.10.1 255.255.255.0
tunnel source Loopback0
tunnel mode gre multipoint
ip nhrp network-id 1
ip nhrp map multicast dynamic
ip nhrp redirect
ip summary-address rip 0.0.0.0 0.0.0.0
```

Listing 18: DMVPN Phase 3 hub tunnel configuration.

The spoke routers R2 and R3 were configured with multipoint GRE tunnels, a static NHRP mapping to the hub, and NHRP shortcut support:

```

interface Tunnel0
 ip address 10.10.10.x 255.255.255.0
 tunnel source Loopback0
 tunnel mode gre multipoint
 ip nhrp map 10.10.10.1 1.1.1.1
 ip nhrp map multicast 1.1.1.1
 ip nhrp nhs 10.10.10.1
 ip nhrp network-id 1
 ip nhrp shortcut

```

Listing 19: DMVPN Phase 3 spoke tunnel configuration.

RIPv2 was configured in the overlay network. On the hub, the networks 192.168.1.0/24 and 10.10.10.0/24 were advertised. On each spoke, the local private network and the DMVPN overlay network were advertised.

2.3 Spoke-to-spoke connectivity test

After configuring DMVPN Phase 3, connectivity between the two spoke sites was tested with a ping from PC2 to PC3.

```

PC2> ping 192.168.3.100

84 bytes from 192.168.3.100 icmp_seq=1 ttl=61 time=36.828 ms
84 bytes from 192.168.3.100 icmp_seq=2 ttl=62 time=15.326 ms
84 bytes from 192.168.3.100 icmp_seq=3 ttl=62 time=17.192 ms
84 bytes from 192.168.3.100 icmp_seq=4 ttl=62 time=17.085 ms
84 bytes from 192.168.3.100 icmp_seq=5 ttl=62 time=17.111 ms

```

Listing 20: Ping output showing the DMVPN Phase 3 shortcut behavior.

The ping was successful, confirming that the two spoke sites were able to communicate through the DMVPN overlay. The first packet had a higher delay and a lower TTL value than the following packets. This is consistent with the initial traffic being sent through the hub before the direct spoke-to-spoke path was established. After the shortcut was created, the following packets used a more direct path between the spokes.

2.4 NHRP and DMVPN state

The DMVPN state was verified using the command:

```
show dmvpn
```

Listing 21: Command used to verify the DMVPN state.

On the hub router R1, two dynamic NHRP entries were observed, one for each spoke:

```

R1# show dmvpn

Interface: Tunnel0, IPv4 NHRP Details
Type:Hub, NHRP Peers:2,

# Ent  Peer NBMA Addr Peer Tunnel Add State  UpDn Tm Attrb
-----
  1 2.2.2.2          10.10.10.2  UP 00:00:32  D
  1 3.3.3.3          10.10.10.3  UP 00:00:05  D

```

Listing 22: DMVPN state observed on the hub router.

This output confirms that the hub dynamically learned both spoke routers through NHRP. The NBMA addresses of the spokes are 2.2.2.2 and 3.3.3.3, and their overlay tunnel addresses are 10.10.10.2 and 10.10.10.3.

On R2, the NHRP cache contained the static entry for the hub:

```
R2# show dmvpn

Interface: Tunnel0, IPv4 NHRP Details
Type:Spoke, NHRP Peers:1,

# Ent  Peer NBMA Addr Peer Tunnel Add State  UpDn Tm Attrb
-----
  1 1.1.1.1              10.10.10.1    UP 00:00:28    S
```

Listing 23: DMVPN state observed on spoke R2.

The same behaviour was observed on R3:

```
R3# show dmvpn

Interface: Tunnel0, IPv4 NHRP Details
Type:Spoke, NHRP Peers:1,

# Ent  Peer NBMA Addr Peer Tunnel Add State  UpDn Tm Attrb
-----
  1 1.1.1.1              10.10.10.1    UP 00:00:08    S
```

Listing 24: DMVPN state observed on spoke R3.

The entries on the spokes are marked as static before spoke-to-spoke resolution because each spoke was manually configured with the hub mapping. On the hub, the entries are dynamic because the spokes register themselves with the hub using NHRP.

After traffic was generated between PC2 and PC3, the NHRP state changed on the spokes. R2 learned a dynamic mapping for the remote spoke R3, associating the overlay address 10.10.10.3 with the NBMA address 3.3.3.3. R3 learned the reverse mapping for R2, associating 10.10.10.2 with 2.2.2.2. These dynamic entries are the practical result of NHRP resolution and allow subsequent packets to follow the direct spoke-to-spoke path.

The entries required for direct forwarding are therefore the NHRP shortcut route to the remote private network and the NHRP cache entry that maps the remote tunnel address to its NBMA address. On R2 this corresponds to a shortcut route towards 192.168.3.0/24 through 10.10.10.3, together with the mapping 10.10.10.3 → 3.3.3.3. On R3 the reverse logic applies: the shortcut route points towards 192.168.2.0/24 through 10.10.10.2, and the NHRP mapping is 10.10.10.2 → 2.2.2.2. These route and cache entries are what remove the hub from the data path after the initial packets.

2.5 Interpretation of the initial and subsequent ICMP packets

During the ping from PC2 to PC3, the first ICMP reply had `ttl=61`, while the following replies had `ttl=62`. This indicates that the initial traffic followed a longer path, through the hub, while the following packets used a shorter spoke-to-spoke path.

This behaviour is expected in DMVPN Phase 3. Initially, the spoke sends the packet towards the hub. The hub then redirects the spoke and allows a direct shortcut to be installed. After this process, the spoke can send traffic directly to the other spoke through the DMVPN overlay, without keeping the hub in the data path.

2.6 RIP and OSPF packets in the public network

In DMVPN Phase 3, two routing protocols were observed in the public network. OSPF was used in the underlay network, while RIPv2 was used in the overlay network.

A Wireshark capture was performed in the public network using the filter:

```
ospf || rip
```

Listing 25: Wireshark filter used for OSPF and RIP traffic.

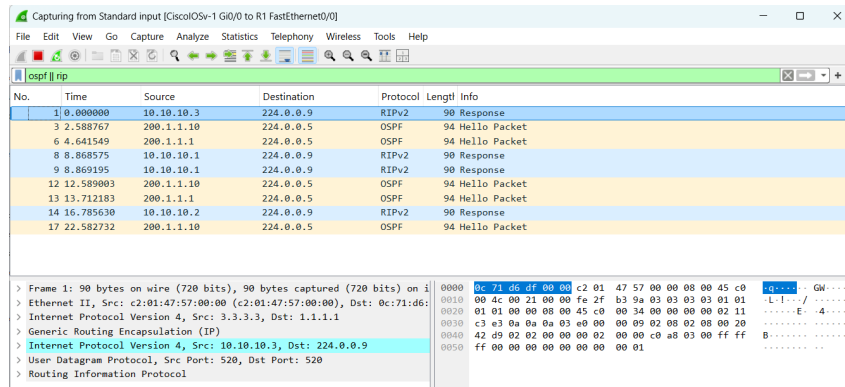


Figure 25: RIP and OSPF packets observed in the public network.

As shown in Figure 25, the OSPF packets belong to the public underlay network. These packets are exchanged directly between public interfaces, using addresses such as 200.1.1.1 and 200.1.1.10, and are sent to the multicast address 224.0.0.5.

The RIPv2 packets belong to the DMVPN overlay network. In the selected packet, the outer IP header uses the loopback/NBMA addresses 3.3.3.3 and 1.1.1.1. The packet is then encapsulated in GRE, and the inner IP header shows the overlay address 10.10.10.3 sending a RIPv2 update to 224.0.0.9. This confirms that RIP is transported through the DMVPN tunnel, while OSPF is used only to provide reachability in the public underlay.

2.7 Role of ip summary-address rip

In the DMVPN Phase 3 configuration, the hub router R1 was configured with the following command under the tunnel interface:

```
ip summary-address rip 0.0.0.0 0.0.0.0
```

Listing 26: RIP summary route command configured on the hub.

This command makes the hub advertise a default route to the spokes through RIP. As a result, R2 and R3 do not need to learn all individual private networks through RIP. Instead, they can send traffic for remote private networks towards the hub using the default route.

The routing table of R2 was analysed with:

```
show ip route
```

Listing 27: Command used to inspect the routing table.

```

Gateway of last resort is 10.10.10.1 to network 0.0.0.0

R*  0.0.0.0/0 [120/1] via 10.10.10.1, 00:00:24, Tunnel0
    1.0.0.0/32 is subnetted, 1 subnets
O    1.1.1.1 [110/12] via 200.2.2.10, 00:08:52, GigabitEthernet0/0
    2.0.0.0/32 is subnetted, 1 subnets
C    2.2.2.2 is directly connected, Loopback0
    3.0.0.0/32 is subnetted, 1 subnets
O    3.3.3.3 [110/3] via 200.2.2.10, 00:09:02, GigabitEthernet0/0
    10.0.0.0/8 is variably subnetted, 3 subnets, 2 masks
C    10.10.10.0/24 is directly connected, Tunnel0
L    10.10.10.2/32 is directly connected, Tunnel0
H    10.10.10.3/32 is directly connected, 00:07:42, Tunnel0
    192.168.2.0/24 is variably subnetted, 2 subnets, 2 masks
C    192.168.2.0/24 is directly connected, GigabitEthernet0/1
L    192.168.2.2/32 is directly connected, GigabitEthernet0/1
H    192.168.3.0/24 [250/255] via 10.10.10.3, 00:07:42, Tunnel0
O    200.1.1.0/24 [110/11] via 200.2.2.10, 00:09:12, GigabitEthernet0/0
    200.2.2.0/24 is variably subnetted, 2 subnets, 2 masks
C    200.2.2.0/24 is directly connected, GigabitEthernet0/0
L    200.2.2.2/32 is directly connected, GigabitEthernet0/0
O    200.3.3.0/24 [110/2] via 200.2.2.10, 00:09:12, GigabitEthernet0/0
R2#

```

Figure 26: Routing table of R2 with the RIP default route received from the hub.

The routing table shows that the spoke received a default route through RIP. This route initially points traffic towards the hub. When traffic is sent to another spoke, NHRP redirects and shortcuts can then install a more direct path to the remote spoke.

The purpose of the summary command is therefore to simplify the routing information advertised to the spokes and to make the hub act as the initial next hop for unknown remote networks.

2.8 RIP split horizon

The command `ip summary-address rip 0.0.0.0 0.0.0.0` was temporarily removed from the tunnel interface of R1 in order to observe the behaviour of RIP without the default route summarization.

```

interface Tunnel0
no ip summary-address rip 0.0.0.0 0.0.0.0

```

Listing 28: Command used to remove the RIP summary route.

After removing the command, the RIP packets and the routing tables were analysed again. Due to RIP split horizon, routes learned by the hub from one spoke through the tunnel interface are not advertised back out of the same tunnel interface to the other spokes. This prevents a spoke from learning the private networks of the other spokes through normal RIP advertisements from the hub.

The behaviour was then compared with split horizon disabled on the hub tunnel interface:

```

interface Tunnel0
no ip split-horizon

```

Listing 29: Command used to disable RIP split horizon on the hub tunnel.

With split horizon disabled, R1 can advertise routes learned from one spoke back out of the same multipoint tunnel interface towards the other spoke. This allows the spokes to learn more specific remote routes through RIP. However, the Phase 3 design used in the lab intentionally keeps the hub summary route and relies on NHRP redirects and shortcuts to build direct spoke-to-spoke forwarding after the first packets.

This behaviour explains why the summary route is important in this design. By advertising a default route, the hub avoids the need to advertise each spoke route individually back through the same multipoint tunnel interface.

After this test, the command was reinstalled on R1:

```
interface Tunnel0
 ip summary-address rip 0.0.0.0 0.0.0.0
```

Listing 30: Command used to restore the RIP summary route.

2.9 NHRP redirects, shortcuts and ICMP traffic

To analyse the behaviour of DMVPN Phase 3 during spoke-to-spoke communication, a Wireshark capture was performed in the public network while PC2 was pinging PC3.

```
PC2> ping 192.168.3.100
```

Listing 31: Ping command used for spoke-to-spoke communication.

The capture was performed using filters for NHRP and ICMP traffic:

```
ip.proto == 47 || icmp
```

Listing 32: Wireshark filter used for GRE and ICMP traffic.

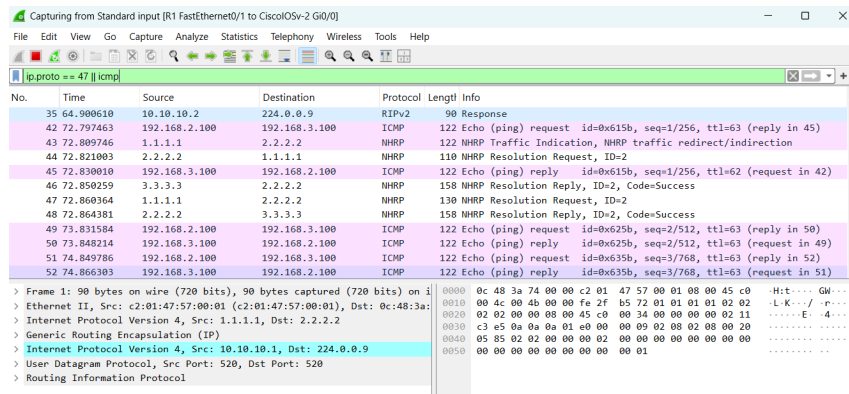


Figure 27: NHRP and ICMP packets during spoke-to-spoke communication.

As shown in Figure 27, the first ICMP Echo Request was sent from PC2 to PC3. Since R2 did not initially have a direct mapping to R3, the first traffic was sent through the hub. The capture then shows NHRP messages, including a traffic indication/redirect and NHRP resolution messages.

The NHRP Resolution Request and Resolution Reply allow the spoke to discover the NBMA address of the remote spoke. After this process, the following ICMP packets are sent directly between R2 and R3, without keeping the hub in the data path.

This behaviour is also visible in the ping output: the first reply had a higher delay and a lower TTL value, while the following replies had lower delay and a higher TTL value. This confirms that DMVPN Phase 3 initially uses the hub and then creates a direct spoke-to-spoke shortcut.

2.10 Comparison with AI explanation

An AI tool was asked to explain the roles of RIP, NHRP redirects, NHRP shortcuts and the command `ip summary-address rip` in DMVPN Phase 3. The explanation predicted that RIP would be used to advertise routing information in the overlay, while the hub would advertise a summarized/default route to the spokes. It also predicted that NHRP redirects and shortcuts would allow the spokes to move from an initial hub-based path to a direct spoke-to-spoke path.

The experimental results matched this explanation. The Wireshark capture showed RIPv2 packets transported inside GRE, while OSPF packets belonged to the public underlay network. The routing table of R2 showed a RIP default route via the hub and, after the ping to PC3, an NHRP route to the remote spoke network. The NHRP and ICMP capture also showed the exchange of NHRP messages during the creation of the direct spoke-to-spoke path.

Therefore, the experimental behaviour confirmed the expected operation of DMVPN Phase 3.

2.11 Conclusion

The DMVPN Phase 3 experiment confirmed that the hub dynamically learns the spokes through NHRP, while the spokes keep a static mapping to the hub. OSPF was used only in the public underlay network to provide reachability between the loopback addresses of the routers. RIPv2 was used in the DMVPN overlay to exchange private routing information.

The command `ip summary-address rip 0.0.0.0 0.0.0.0` allowed the hub to advertise a default route to the spokes. This made the spokes initially forward remote traffic through the hub. However, when PC2 communicated with PC3, NHRP redirects and shortcuts allowed a direct spoke-to-spoke path to be created.

Compared with DMVPN Phase 1, Phase 3 avoids keeping the hub permanently in the data path for spoke-to-spoke communication. Compared with DMVPN Phase 2, Phase 3 works better with summarized routing information because NHRP redirects and shortcuts dynamically create direct paths between spokes.